



Copy Fail exploit detection in Microsoft Azure

Computer Forensics and Cyber Crime Analysis
Politecnico di Torino

Chiara Iorio
s343732

May 21, 2026

Contents

1	The context	2
2	The Copy Fail vulnerability	2
2.1	Copy Fail in a Linux system	2
2.2	Copy Fail in Kubernetes	3
2.2.1	Container images	3
2.2.2	Privileged DaemonSet	3
2.2.3	Payload	4
2.2.4	Page cache corruption	5
3	Azure Monitor Logs	6
3.1	Grafana dashboard	6
3.2	AKS monitoring	7
4	Tetragon monitoring	9
4.1	eBPF	9
4.2	Tetragon	10
4.2.1	What it is	10
4.2.2	Its strengths	11
4.2.3	How it works	13
4.2.4	Integration with cloud providers native monitoring	13
4.3	Tetragon logs	15
4.3.1	Default monitoring	15
4.3.2	Filesystem mount monitoring	15
4.3.3	Socket binding monitoring	17
4.3.4	Blocking the exploit	18
5	Forensic considerations	19
6	Conclusions	20

1 The context

This work falls under the scope of corporate cloud forensics. We consider a software company, where multiple teams share a multi-tenant Kubernetes cluster to test the applications they are developing. Even though there is some level of trust between members of different teams, namespaces, Role Based Access Control, quotas, and network policies are used to safely and fairly share clusters. For example, employees can only create unprivileged pods, to follow the least privilege paradigm and reduce the attack surface. The company is relying on the Microsoft Azure [1] cloud computing platform, and in particular on the Azure Kubernetes Service (AKS) to provision the cluster: this choice simplifies the service management, as Azure takes care of the control plane and critical operations like health monitoring and maintenance. On 29th April 2026, the Copy Fail vulnerability is publicly disclosed, and we want to find traces of the possible exploit performed by an employee to do container escape and access to the underlying physical node. In this analysis, we rely both on the native logs provided by Azure Monitor and on those collected by an eBPF-based tool like Tetragon.

2 The Copy Fail vulnerability

2.1 Copy Fail in a Linux system

The CVE-2026-31431 (Copy Fail) [2] allows for a 100% reliable local privilege escalation on every Linux distribution shipped from 2017 to April 2026. It is classified with a CVSS Score of 7.8 (high) and *Important* severity. It is a logic bug in the Linux kernel's `authenc:esn` cryptographic template, an AEAD wrapper used by IPsec for Extended Sequence Number (ESN) support, that allows an unprivileged local user to trigger a deterministic, controlled 4-byte write into the page cache of any readable file on the system. The page cache stores file-backed memory pages, including file contents read from disk, file contents written but not yet flushed, executable binaries, shared libraries, and memory-mapped files. A single 732-byte Python script can edit a `setuid` binary (e.g., `/usr/bin/su`) and obtain full root privileges. Because only the in-memory page cache is corrupted, standard file integrity tools miss the exploit as the on-disk file remains unchanged.

2.2 Copy Fail in Kubernetes

In Kubernetes, the same vulnerability manifests differently, allowing both for cross-container lateral movement and container escape. We focus on the second case, in which a fully unprivileged container can achieve node-level code execution. The PoC and discussion are inspired by [3, 4].

2.2.1 Container images

To better understand how the exploit works in this context, some preliminary knowledge on how container images work is needed [5]. A container image is not a single file, but a stack of layers, where each one adds changes on top of the previous one. The base layer contains the operating system: system binaries (e.g., `/usr/bin/cat` and `/usr/bin/bash`), shared libraries (e.g., `libc.so.6`), and other fundamental tools. When multiple images share the same base layer, by design Kubernetes stores it only once on the node, to save disk space and speeds up image pulls. When containers run on the same node, the container runtime (*containerd*) uses a filesystem called *overlayfs* to assemble each container’s view of the filesystem. Under the hood, the base layer files are the same physical data on disk and the same in-memory copy in the Linux page cache. When the kernel reads a file from disk, it keeps a copy in memory called the page cache: this speeds up future reads since the kernel can serve the file from memory instead of hitting the disk again. The page cache is a node-wide, shared resource, not isolated per container. If two containers share the same base layer, they read from the same page cache entry. In normal operation on an *overlayfs* mount, opening a lower-layer file for write access or truncation triggers a copy-up before writes proceed, allocating a new inode in the pod’s upper layer so the change is private. By storing only this small set of differences, containers can reuse their lower layers efficiently while still allowing a writable copy to be presented to applications. However, Copy Fail skips the standard write path entirely: the page cache it mutates belong to the lower-layer `address_space` itself, shared host-wide, rather than the upper layers that were meant to store write deltas.

2.2.2 Privileged DaemonSet

Any privileged DaemonSet sharing image layers with an attacker-controlled container can be weaponized for container escape. This PoC uses *kube-proxy* as one concrete example, a critical Kubernetes network component that runs with high privileges on the host system. This technique generalizes to any privileged workload on the cluster (e.g., monitoring agents, CNI plugins, log

collectors). In the following listing, we see the Dockerfile used to create a container image matching the *kube-proxy* version in AKS:

```
FROM fedora:43 AS busybox
RUN dnf install -y busybox

FROM mcr.microsoft.com/oss/v2/kubernetes/
kube-proxy:v1.34.4-1 AS kube-proxy
COPY --from=busybox /bin/busybox.musl.static /bin/busybox

RUN ["/bin/busybox", "--install", "-s", "/bin"]
COPY bin/copyfail /bin/copyfail
```

First, a temporary Fedora container is used as a package source to download the *busybox* package that packs tiny versions of many common utilities (e.g., *sh*, *ls*, *cat*) into a single small executable. It is injected into the final *kube-proxy* image to provide a command-line shell and basic tools. This is necessary because production Kubernetes images are often “distroless”, meaning they contain zero troubleshooting tools or shells by default. The files are copied into *kube-proxy*: to make the PoC work, we need the exact version of the image used and not a general *kube-proxy* one. The final line copies the binary *copyfail* into the container. In this setup, BusyBox provides the necessary shell environment to interact with the container, navigate the system, and execute that binary. The binary (embedding the attack C payload) must be built for the specific node architecture (discoverable running *uname -r*).

When the unprivileged PoC container corrupts a binary’s page cache exploiting the Copy Fail vulnerability, the corruption is immediately visible to the privileged container on the same node, with zero cross-container communication. Next time the privileged container executes the targeted binary (i.e., */usr/sbin/ipset*), it will execute the payload injected by the attacker. Fig. 1 exemplifies the previous discussion.

2.2.3 Payload

The attacker can inject the payload they want inside the target binary cache. In this example, the injected payload is the following:

```
#include "nolibc/nolibc.h"

int main(void) {
    const char msg[] = "CONTAINER ESCAPE SUCCESSFUL!\n";
    int fd;

    mkdir("/mnt", 0755);
```



Figure 1: Page cache corruption due to image layer sharing.

```

mount("/dev/sda1", "/mnt", "ext4", 0, NULL);

fd = open("/mnt/root/res", O_WRONLY|O_CREAT|O_TRUNC,
0644);
write(fd, msg, sizeof(msg) - 1);
close(fd);

umount2("/mnt", 2);
return 0;
}

```

This code is built against `nolibc` (a minimal C library) for a static, dependency-free binary. It mounts the host filesystem and creates a file in the `/root/` directory to demonstrate the exploit. Once the attacker has access to the node, they may do whatever is needed to achieve their goal, like spawning a reverse shell. The only constraint is that the compiled code must not exceed the target binary's size.

2.2.4 Page cache corruption

Finally, we examine how the page cache corruption occurs [6]. The kernel's `AF_ALG` subsystem exposes a socket-based interface for userspace cryptographic operations. The exploit is composed of four main phases:

1. **Socket setup:** Open the target binary read-only and bind the `AF_ALG` AEAD socket to `authenc:sn`. `AF_ALG` is available to unprivileged users by default.
2. **Constructing the Payload:** Send a small payload chunk through the socket using `sendmsg()` with the flag `MSG_MORE` set, telling the

kernel to expect more data. Bytes 4–7 of the Additional Authenticated Data contain the 4 B the attacker wishes to write into the target page cache.

3. **Memory Corruption via Copy Fail:** The attacker uses the `splice()` system call to map the page cache of the target binary into the crypto socket. When the attacker calls `recvmsg()` to trigger the decryption process, the kernel incorrectly fails to create a private copy of the data when it is modified in place. This allows the attacker to corrupt the target page cache.
4. **Container Escape:** When any process in the privileged container executes the corrupted binary, the shellcode runs with its original privileges, allowing attackers to break out of isolated Linux containers and compromise the underlying shared kernel.

Notice that write permission to the file is not needed. Moreover, the file on disk remains unchanged, as only the in-memory page cache is corrupted.

3 Azure Monitor Logs

Azure Monitor [7] is Microsoft’s unified observability service for collecting, analyzing, and acting on telemetry from cloud and hybrid environments. It allows to understand the health, performance, and reliability of the Azure applications and infrastructure resources by bringing together metrics, logs, traces, and events into a single observability platform integrated into the Azure portal for each service (e.g., AKS clusters). It has a centralized data platform to support collection of telemetry from a variety of sources and the different features for analysis and troubleshooting. The platform uses Log Analytics workspaces to collect log and trace data, which can be analyzed with Kusto Query Language (KQL), and Azure Monitor workspaces to collect Prometheus (an open monitoring tool) metrics. These metrics can be used to create custom Grafana dashboards [8].

3.1 Grafana dashboard

Fig. 2 shows a custom Grafana dashboard built using the metrics from the managed Prometheus of the AKS cluster. Starting from the top, it illustrates the number of ready nodes in the cluster (i.e., 1) and the number of running pods in each namespace. Specifically, the *kube-system* namespace, managing cluster resources, contains the highest number of pods (29). Then, *tenant-blue*



Figure 2: Custom Grafana dashboard with metrics from the AKS cluster.

and *tenant-green* are namespaces associated to two different teams in the company. We will talk about the *vector* namespace in the subsequent sections. In the middle, we have two plots about the CPU and RAM usages per namespace: the namespaces using most CPU resources (0.8%) are *tenant-blue* and *tenant-green*, probably associated to workloads they are testing. Most RAM, instead, is used by the *kube-system* namespace. In the bottom plots, we see the amount of bytes transmitted and received by each namespace in Mb/s. Again, the most active namespace is *kube-system*. By looking at these plots, we do not notice anything suspicious, like an employee namespace consuming excessive resources.

3.2 AKS monitoring

The native monitoring of the AKS cluster can give us some insights on what is happening in real-time in our clusters. Yet, it cannot be used for a retroactive investigation and, in that case, we need to rely on the logs stored in Log Analytics.

However, if we have the chance to monitor in real-time what is happening, we can look at the list of pods deployed in our cluster and identify any suspicious one. In this case, we identify the pod *copy-fail-poc-76c59fb745-xkm2s*, and look at its details (fig. 3). Specifically, we can determine the container name (*copy-fail-poc*) and ID, the namespace in which it is deployed

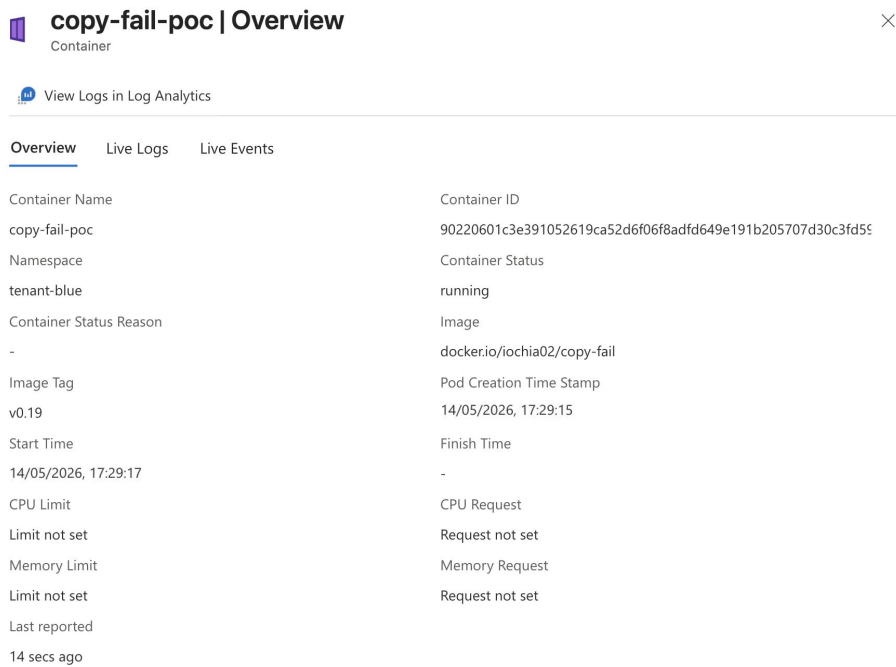


Figure 3: Pod details retrieved from the AKS native monitoring.

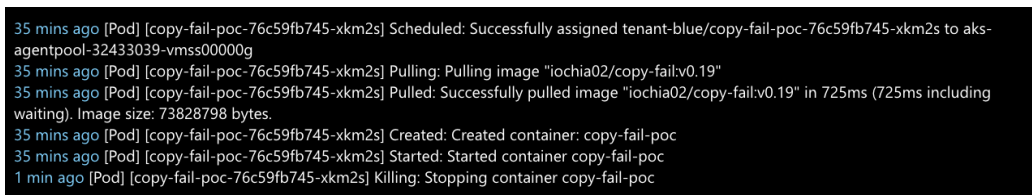


Figure 4: Pod live events from the AKS native monitoring.

(*tenant-blue*), the start time, its status (*running*), and the base image, along with the registry from which it was pulled. In this case, we recognize that a situation is unfolding thanks to the container names that, however, can be easily modified by attackers not to be easily identifiable. The image registry can be more informative: if the image is pulled from a not well-known recognized one we can classify this as a possible attack.

In the same page, we can also see the *Live Events* connected to a specific container (fig. 4). However, we cannot deduce much from these logs, as the last two lines only tell us when the container *copy-fail-poc* started and when it was killed, but we cannot know anything that was executed there.

If, instead, we are running a retroactive investigation, we can rely on the health logs stored in the *KubePodInventory* table in the logs tab of the

5/14/2026, 6:27:57.000 ...	aks-agentpool-32433039-vmss0000...	/subscriptions/0b269113-1590-4ade-8265-6a10772d6764/...	5/14/2026, 4:03:54.000 PM	c8c88c55-832c-4d68-9f2
TenantId	a951ceae8-e540-4234-ad41-694a7583f4ca			
SourceSystem	Containers			
TimeGenerated [UTC]	2026-05-14T18:27:57Z			
Computer	aks-agentpool-32433039-vmss00000g			
ClusterId	/subscriptions/0b269113-1590-4ade-8265-6a10772d6764/resourceGroups/k8s-cluster/providers/Microsoft.ContainerService/managedClusters/cluster-forensics			
ContainerCreationTimeStamp [UTC]	2026-05-14T16:03:54Z			
PodUid	c8c88c55-832c-4d68-9f20-0f6dffa6b98			
PodCreationTimeStamp [UTC]	2026-05-14T16:03:52Z			
ContainerRestartCount	0			
PodRestartCount	0			
PodStartTime [UTC]	2026-05-14T16:03:52Z			
ControllerKind	ReplicaSet			
ControllerName	copy-fail-poc-76c59fb745			
ContainerStatus	running			
ContainerID	90220601c3e391052619ca52d6f068adf649e191b205707d30c3fd597f3a4b			
ContainerName	c8c88c55-832c-4d68-9f20-0f6dffa6b98/copy-fail-poc			
Name	copy-fail-poc-76c59fb745-cqkrd			
> PodLabel	[{"app.kubernetes.io/name":"copy-fail-poc","app.kubernetes.io/part-of":"cve-2026-31431","pod-template-hash":"76c59fb745"}]			
Namespace	tenant-blue			
PodStatus	Running			
ClusterName	cluster-forensics			
PodIp	10.244.0.79			
ContainerLastStatus	0			
Type	KubePodInventory			
_ResourceId	/subscriptions/0b269113-1590-4ade-8265-6a10772d6764/resourceGroups/k8s-cluster/providers/microsoft.containerservice/managedclusters/cluster-forensics			

Figure 5: Log from the *KubePodInventory* table.

AKS cluster page, storing Kubernetes pod and container information. Fig. 5 shows a log relative to the *copy-fail-poc* container. Again, we can retrieve the container and pod names, IDs, namespace, creation time and to which resource they are associated. However, no information about what the container has executed is present.

4 Tetragon monitoring

As we have seen in the previous paragraphs, it is difficult to detect the Copy Fail exploit by simply using the monitoring tools provided by the Azure monitor. To get more precise insights about the events generated by the cluster we can rely on eBPF-based tools like Tetragon, which must be deployed directly in the cluster by the system manager. In this way, the forensic investigator will have more logs to analyze in case of an incident. In the next paragraphs, we deep dive into the involved technologies.

4.1 eBPF

eBPF [9, 10] is a technology that enables event-driven custom code to run natively in an operating system kernel. This custom code has to pass through multiple checks, to avoid that unsafe code is injected into the kernel. Moreover, since the compiler translates the generic bytecode into the architecture specific

CPU code, the performance is as close as possible to native execution. eBPF extends kernel functionalities without requiring changes to applications or the kernel to observe and enforce runtime security policies. eBPF collects and filters security observability data directly in the kernel. Then, these events are sent to userspace and can be shipped to permanent storage for analysis, threat hunting, incident investigation, building out security policies, or to a SIEM for advanced inspection. The main advantage of this technology is that it understands container attributes like namespaces (used by containers to isolate resources), capabilities (defining what a process is allowed to do) and cgroups (used by containers to limit usable CPU and memory resources). Hence, it can provide an identity-aware security observability, that allows you, for example, to trace the full lifecycle of processes, network sockets or pods.

eBPF allows us to retrieve and observe many kinds of events. First, it is possible to monitor system calls, that can help to identify malicious behavior such as reading or writing sensitive files, changing kernel namespace or loading a kernel module. Second, it allows tracking all network connections at the socket layer, to create a least-privilege network policy. Third, it is possible to monitor unauthorized host filesystem access in containers, that can lead to privilege escalation techniques to carry out an attack. Last but not least, we can detect malicious behavior also in our cloud native infrastructure, monitoring standard Linux processes on the underlying host, that can be both a physical or a virtual machine, that runs containerized workloads and standard processes using the same kernel. In this way, we can have a full visibility on the different phases of an attack. Another advantage is that users do not have to write eBPF programs by themselves, but can rely on already existing solutions, like Falco, Tracee and Tetragon.

4.2 Tetragon

4.2.1 What it is

Tetragon [10, 11, 12] is an open-source Kubernetes-native synchronous monitoring, filtering and enforcement tool from the makers of Cilium (an open source software for providing, securing, and observing network connectivity between container workloads) that applies policies and filtering directly in-kernel with eBPF. It captures different process and network event types through a user-supplied configuration to enable security observability on arbitrary hook points into the kernel. Tetragon can detect and react to process execution events, system call activity, and I/O activity including network and file access. These types of events correspond to signals that can be used to detect a suspicious behavior. For example, to detect process execution, Tetragon

detects when a process starts and stops. To monitor network connections, it detects whenever a process opens, closes or listens a network socket. File access is achieved by monitoring file descriptors and system calls such as `open`, `read` and `write`. To get layer 7 network visibility, it takes advantage of the observed fields during a connection via network socket.

There is also an enterprise version of Tetragon [13], which allows to collect additional data that can be used to identify which application is consuming all the bandwidth, spot DNS issues from DNS requests, responses, or name lookups, detect weak TLS cipher versions, and pinpoint the source of latency in an application. However, in the remainder of this document we will focus on the open-source version.

Tetragon can be installed on standalone Linux systems, whether bare metal, virtual machines or as part of a cloud provider machine.

4.2.2 Its strengths

Tetragon has multiple advantages in respect to prior solutions [14, 15, 16, 17], that make it widely adopted.

Kubernetes Native Tetragon is aware of Kubernetes and runs natively in Kubernetes as a DaemonSet, ensuring that all or some nodes run a copy of a pod. All security observability events are automatically enriched with Kubernetes metadata such as pod names, labels, namespace information, and container SHAs. It allows to apply policies to specific pods or namespaces, improving control and reducing alert noise through in-kernel event filtering and aggregation, leading to lower CPU and memory overhead.

Deep observability Tetragon traces events directly from within the kernel, allowing it to see everything at runtime, from system call activity, I/O file access, process execution events, and network activity. Hence, it can detect low-level microbursts in TCP connections, provide HTTP visibility for dashboards, or detect the use of particular vulnerable shared libraries. This is coupled with Kubernetes awareness, which matches security events and policies based on Kubernetes metadata rather than ephemeral IP addresses.

Transparency To use Tetragon, no application code changes are needed, since all observability data is collected transparently from within the kernel. Applications cannot detect when they are being monitored, which is ideal for security.

Low-Overhead Minimal overhead is imposed on the system. Performing filtering, aggregation, metric accounting, and histogram collection directly in the kernel with eBPF helps to reduce the overhead. By limiting the data transferred to userland to only relevant events, Tetragon also removes noise, and eliminates race conditions and unnecessary delay when taking enforcement action. For example, Tetragon process execution tracking adds only a 1.7% overhead and 2.5% overhead when also writes all process execution events as JSON to disk.

Easy to use Using Tetragon for observability is quite simple, also for non-experts. The Policy Library on GitHub provides structured examples of Tetragon policies paired with relevant use cases. From there, you can pick the most suitable for your case and apply it on your system to immediately get a data rich view of what's happening inside your Linux machines and Kubernetes clusters. The policies can be used as-is or as a template for customization before applying them.

Real-Time Enforcement Enforcement of runtime policies happens in-kernel and synchronously (real-time) allowing to prevent attacks instead of reacting to them asynchronously. Tetragon has the ability to specify allow lists for access control at several layers.

Proactive security enforcement and control Tetragon not only allows to detect malicious behaviour, but also to block it, by killing the processes matching specific criteria or by modifying a function's return value, preventing it from being executed.

Deploy dynamic policies for zero days With Tetragon, policies are not static. For a zero-day attack, when a new vulnerability is disclosed, you can immediately deploy a Tetragon policy to the entire production environment without requiring a restart or downtime.

Embedded zero trust across Linux infrastructure Zero trust usually involves juggling numerous tools, each one addressing different aspects of security (identity, network, OS, service mesh, ...), resulting in a fragmented and complex control over the environment. Tetragon solves this fragmented approach, bringing these layers together and giving a unified method to collecting, observing, and enforcing on security-significant events.

Defense in depth By having multiple layers of observability and enforcement policies across different stages, malicious actors have to scale through multiple layers of security controls. For example, if one tracing policy is unable to prevent initial access to the environment, another one can prevent executions or privilege escalations.

SIEM integration Tetragon events can be exported as JSON files, allowing to integrate modern observability and policy standards such as Prometheus, fluentd, OpenTelemetry, Open Policy Agent, and traditional SIEM platforms.

Streamline compliance attestation Tetragon can be used to enforce policies at runtime, alert on critical security-significant events, and collect health metrics for known compliance standards, including SOC2, PCI-DSS, NIST-800, and more. Hence, you can build proactive policies and verifiable attestation pipelines to ensure that every software package is delivered exactly as the author intended, free from malicious interference and exploits of supply-chain vulnerabilities.

4.2.3 How it works

Tetragon functioning is based on TracingPolicies [18]. A TracingPolicy is a user-configurable Kubernetes custom resource that allows users to trace arbitrary events in the kernel and optionally define actions to take on a match. They can be applied also to a subset of pods, running in a specific namespace: in this case it is declared as NamespacedTracingPolicy. We will see examples of policies in the next paragraphs.

4.2.4 Integration with cloud providers native monitoring

Tetragon logs can be integrated with those natively captured by cloud providers tools. Specifically, we investigate how to integrate them with Azure’s Log Analytics.

Fig. 6 illustrates the monitoring pipeline. Tetragon generates JSON-formatted event logs and stores them in a file (`/var/run/cilium/tetragon`) on the node. Vector [19], a lightweight, high-performance data pipeline tool used to collect, transform, and route logs, metrics, and traces, parses, filters, and routes these events to an endpoint, namely a *sink*. In this context, we use the Azure Logs Ingestion sink, that uses the Azure Monitor Logs Ingestion API to send log events to the Log Analytics Workspace. Then, we need to configure different components directly on the Azure portal [20].

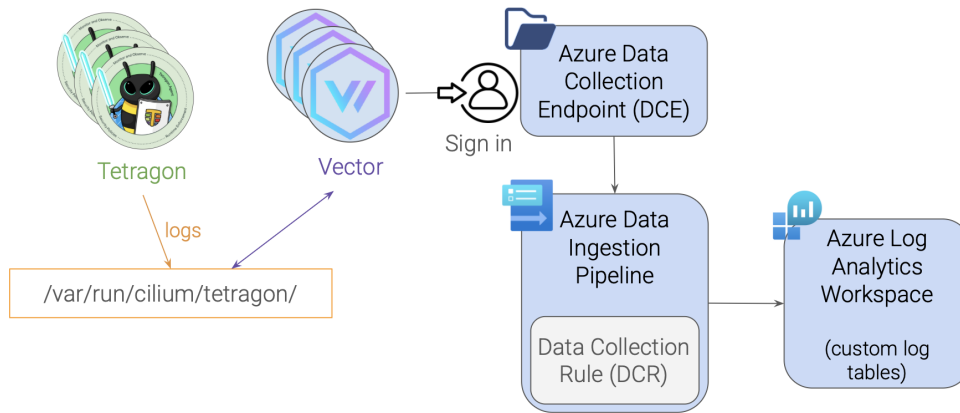


Figure 6: The monitoring pipeline.

App registration in Microsoft Entra ID It is needed to create a client secret, a client ID and a tenant ID. A client secret is a secret known only to your application and the authorization server, used to authenticate your application to the authorization server when your application requests an access token. It is suitable to test environments, while for production ones it is better to move to a client certificate.

Data Collection Endpoint (DCE) The physical URL to which Vector sends the logs. It receives the authenticated HTTPS request thanks to the client secret (directly inserted in the Vector manifest).

Custom Table in the Log Analytics Workspace It is the table we will be sending data to. The columns are gotten from an example of JSON logs we have to provide at creation time. The table we create here is *Copyfail_CL*.

Data collection rule (DCR) It defines how the data will be handled once it's received. This includes the schema of data that's being sent to the endpoint, the transformation that will be applied to the data before it's sent to the workspace (in our case, none) and the destination workspace and table the transformed data will be sent to.

Once everything is properly configured, the logs will directly be accessible from the Log Analytics interface on the Azure portal.

> node_labels	["agentpool":"agentpool","beta.kubernetes.io/arch":"amd64","beta.kubernetes.io/instance-type":"Standard_D2as_v4","beta.kubernetes.io/os":"linux","failure-domain.beta.kubernetes.io/region":"austriaeast"]		
node_name	aks-agentpool-32433039-vmss00000g		
> process_exec	["parent":{"aud":"4294967295","binary":"/bin/bash","cwd":"/","docker":"ae3126fdc30e4054eab60cea7161be3","exec_id":"YWtzLWFnZW50cG9vbC0zMjQzMzAzOS12bXNzMdAwMDN0JExMzQ5ODQwNjI2MDU6MzkmMTE=","uid":"4294967295","binary":"/bin/bash","cwd":"/","docker":"ae3126fdc30e4054eab60cea7161be3","exec_id":"YWtzLWFnZW50cG9vbC0zMjQzMzAzOS12bXNzMdAwMDN0JExMzQ5ODQwNjI2MDU6MzkmMTE=","arguments":"--target /usr/sbin/ipset","uid":"4294967295","binary":"/bin/copyfail","cwd":"/","docker":"ae3126fdc30e4054eab60cea7161be3","exec_id":"YWtzLWFnZW50cG9vbC0zMjQzMzAzOS12bXNzMdAwMDN0JExMzQ5ODQwNjI2MDU6MzkmMTE="}		
> parent	["aud":"4294967295","binary":"/bin/bash","cwd":"/","docker":"ae3126fdc30e4054eab60cea7161be3","exec_id":"YWtzLWFnZW50cG9vbC0zMjQzMzAzOS12bXNzMdAwMDN0JExMzQ5ODQwNjI2MDU6MzkmMTE=","uid":"4294967295","binary":"/bin/bash","cwd":"/","docker":"ae3126fdc30e4054eab60cea7161be3","exec_id":"YWtzLWFnZW50cG9vbC0zMjQzMzAzOS12bXNzMdAwMDN0JExMzQ5ODQwNjI2MDU6MzkmMTE=","arguments":"--target /usr/sbin/ipset","uid":"4294967295","binary":"/bin/copyfail","cwd":"/","docker":"ae3126fdc30e4054eab60cea7161be3","exec_id":"YWtzLWFnZW50cG9vbC0zMjQzMzAzOS12bXNzMdAwMDN0JExMzQ5ODQwNjI2MDU6MzkmMTE="}		
> process	["arguments":"--target /usr/sbin/ipset","uid":"4294967295","binary":"/bin/copyfail","cwd":"/","docker":"ae3126fdc30e4054eab60cea7161be3","exec_id":"YWtzLWFnZW50cG9vbC0zMjQzMzAzOS12bXNzMdAwMDN0JExMzQ5ODQwNjI2MDU6MzkmMTE="}		
arguments	--target /usr/sbin/ipset		
uid	4294967295		
binary	/bin/copyfail		
cwd	/		
docker	ae3126fdc30e4054eab60cea7161be3		
exec_id	YWtzLWFnZW50cG9vbC0zMjQzMzAzOS12bXNzMdAwMDN0JExODYyMDgwNDQwNzY6Mzk4NzQ=		
flags	execve rootcwd clone		
in_init_tree	false		
parent_exec_id	YWtzLWFnZW50cG9vbC0zMjQzMzAzOS12bXNzMdAwMDN0JExMzQ5ODQwNjI2MDU6MzkmMTE=		
pid	39874		
> pod	["container":{"id":"containerd://ae3126fdc30e4054eab60cea7161be393f98a54a996df44d6b815ab60f53ca5b","image":{"id":"docker.io/occhia02/copy-fail@sha256:48e611d0b6b1d8-fa14-48e9-813a-1b8a22fce03e","name":"copy-fail-poc-76c59fb745-xkm2s","namespace":"tenant-blue","pod_labels":{"app.kubernetes.io/name":"copy-fail-poc"},"uid":"db6eb1d8-fa14-48e9-813a-1b8a22fce03e","workload":"copy-fail-poc"},"start_time":2026-05-14T15:36:10.9954393Z,"tid":39874,"uid":0}		
start_time	2026-05-14T15:36:10.9954393Z		
tid	39874		
uid	0		
TimeGenerated [UTC]	2026-05-14T15:36:11.767611Z		
TenantId	a951cea8-e540-4234-ad41-694a7583f4ca		
Type	CopyFail_CL		

Figure 7: Log relative to the execution of `/bin/copyfail`.

4.3 Tetragon logs

4.3.1 Default monitoring

Without any explicit policies, Tetragon automatically tracks all process lifecycle events across pods, including process executions and exits. Hence, we can first look at the logs in our custom table (i.e., *Copyfail_CL*) and see if we identify anything suspicious. An example of log is reported in fig. 7.

Specifically, we can retrieve some useful information: the command `/bin/copyfail` with arguments `-target /usr/sbin/ipset` was run by the container `copy-fail-poc-76c59fb745-xkm2s` in the namespace `tenant-blue`. It specifies also the registry from which the image was originally pulled. Moreover, other information are present: the timestamp at which the event was generated, the node specifications, the parent process, all the details about the process itself and the pod. Obviously, this log can help us in identifying the attack as the binary name is quite self-explanatory, but we would expect an expert attacker to change it. The same considerations done previously on the image registry still apply.

4.3.2 Filesystem mount monitoring

For this reason, the cluster manager can decide to deploy specific policies to detect suspicious actions performed by employees. Hence, a *TracingPolicy* to monitor filesystem mounts can be added, as these operations represent the primary mechanism used by attackers to escape a container and compromise the underlying host node. For example, they can deploy the following policy:


```

apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "monitor-mounts"
spec:
  kprobes:
    - call: "sys_mount"
      syscall: true
      args:
        - index: 0
          type: "string"
        - index: 1
          type: "string"
        - index: 2
          type: "string"
      selectors:
        - matchActions:
            - action: Post

```

The purpose of this specific policy is to intercept and log every single filesystem mount operation (`sys_mount`) happening inside the Kubernetes cluster at the Linux kernel level. The top section simply gives a recognizable name within Kubernetes. It registers the configuration with the security engine, so it can be managed, updated, or removed like any other standard cluster resource. The `kprobe` part tells the security engine exactly where to place its sensor. It hooks directly into the core operating system function responsible for mounting filesystems. Every time any process (whether it is a legitimate system service or a compromised container) asks the operating system to link a storage source to a folder, this hook is triggered. The `arg` section tells Tetragon which arguments to extract when the mount system call is used and how to parse them:

- Index 0 (Source): The device or directory being mounted.
- Index 1 (Target): The destination directory where the filesystem will be attached (the mount point).
- Index 2 (Filesystem Type): The format type being used (e.g., ext4, xfs, tmpfs).

The final section controls the timing and scope of the alert. It tells the sensor to wait until the operating system has finished executing the request before writing the log. This timing ensures that you only get notified about successful actions, preventing false alarms from failed or denied requests.


```

- matchArgs:
- index: 1
  operator: "Family"
  values:
- "38"          #AF_ALG
matchActions:
- action: Post
  rateLimit: 1s

```

Specifically, it hooks into the kernel security function `security_socket_bind`, capturing details about the socket involved and the address to which it binds. It checks if the Address Family integer (the unique identifier associated to every network communication protocol in Linux) is equal to the value 38, that is the hardcoded kernel constant that represents `AF_ALG` (the Linux Kernel Crypto API User Interface). Finally, it simply reports one log of this type per second to prevent log flooding.

Fig. 9 reports the logs about the usage of the `security_socket_bind` function with the argument `AF_ALG` by the process `/bin/copyfail` in the container `copy-fail-poc-76c59fb745-xkm2s`.

4.3.4 Blocking the exploit

Once we are aware of the presence of the vulnerability, we would like to prevent its exploitation [21]. The safer way is to apply the distribution's kernel update and reboot. If immediate kernel patching and rebooting are not feasible, the attack surface can be eliminated by disabling the `algif_aead` kernel module:

```

echo "install algif_aead /bin/false" > /etc/modprobe.d/
disable-algif-aead.conf rmmod algif_aead 2>/dev/null

```

This is the mitigation deployed on AKS to immediately respond to the vulnerability. However, there are other options. As already seen in §4.2.2, another solution is to leverage on Tetragon, as it allows blocking specific operations directly at kernel level, like the invocation of `security_socket_bind` with `family = 38`. Specifically, we have to modify the last part of the previous TracingPolicy as follows:

```

matchActions:
- action: Sigkill

```

In this case, the logs will show that the function was called but then a `SIGKILL` was sent, blocking the vulnerability exploit.

> node_labels	({"agentpool":"agentpool","beta.kubernetes.io/arch":"amd64","beta.kubernetes.io/instance-type":"Standard_D2as_v4","beta.kubernetes.io/os":"linux","failure-domain.beta.kubernetes.io/region":"australiaeast","failure-domain.beta.kubernetes.io/zone":"australiaeast-1"}
node_name	aks-agentpool-32433039-vmss00000g
TimeGenerated [UTC]	2026-05-14T15:47:19.735606Z
> process_kprobe	({"action":"KPROBE_ACTION_POST","args":{"sockaddr_arg":{"addr":"invalid IP","family":"AF_ALG"},"function_name":"security_socket_bind","parent":{"aid":"4294967295","binary":"/bin/bash","cwd":"/","docker":"ae3126f3d30e4054eab60cea7161be3"},"exec_id":"YWtZLWFhZW50G9vbc0zMjQzMzAzOS12bXNzMDAwMD8nOjE4NTMzOTg4NzUwNTY1NTg=","flags":"exec"},"policy_name":"alg_bind2","process":{"arguments":["--target /usr/sbin/psnet","aid":"4294967295","binary":"/bin/copyfail","cwd":"/","docker":"ae3126f3d30e4054eab60cea7161be3"},"exec_id":"YWtZLWFhZW50G9vbc0zMjQzMzAzOS12bXNzMDAwMD8nOjE4NTMzOTg4NzUwNTY1NTg=","pid":56558,"pod":{"container":{"id":"containerd://ae3126f3d30e4054eab60cea7161be39398a54a996df44d6b815ab0f53ca5b","image":{"id":"docker.io/copy-fail@sha256:48e61d8fa1448e9813a1b8a22fce03e"},"name":"copy-fail-poc","namespace":"tenant-blue"},"pod_labels":{"app.kubernetes.io/name":"copy-fail-poc","db6eb1d8-fa14-48e9-813a-1b8a22fce03e"},"workload":"copy-fail-poc"},"refcnt":1,"start_time":2026-05-14T15:47:18.1862700Z,"tid":56560,"uid":0,"return_action":"KPROBE_ACTION_POST"}
action	KPROBE_ACTION_POST
args	[{"sockaddr_arg":{"addr":"invalid IP","family":"AF_ALG"}}]
function_name	security_socket_bind
parent	({"aid":"4294967295","binary":"/bin/bash","cwd":"/","docker":"ae3126f3d30e4054eab60cea7161be3"},"exec_id":"YWtZLWFhZW50G9vbc0zMjQzMzAzOS12bXNzMDAwMD8nOjE4NTMzOTg4NzUwNTY1NTg=","flags":"exec","policy_name":"alg_bind2"}
process	({"arguments":["--target /usr/sbin/psnet","aid":"4294967295","binary":"/bin/copyfail","cwd":"/","docker":"ae3126f3d30e4054eab60cea7161be3"},"exec_id":"YWtZLWFhZW50G9vbc0zMjQzMzAzOS12bXNzMDAwMD8nOjE4NTMzOTg4NzUwNTY1NTg=","pid":56558,"pod":{"container":{"id":"containerd://ae3126f3d30e4054eab60cea7161be39398a54a996df44d6b815ab0f53ca5b","image":{"id":"docker.io/copy-fail@sha256:48e61d8fa1448e9813a1b8a22fce03e"},"name":"copy-fail-poc","namespace":"tenant-blue"},"pod_labels":{"app.kubernetes.io/name":"copy-fail-poc","db6eb1d8-fa14-48e9-813a-1b8a22fce03e"},"workload":"copy-fail-poc"},"refcnt":1,"start_time":2026-05-14T15:47:18.1862700Z,"tid":56560,"uid":0,"return_action":"KPROBE_ACTION_POST"}
arguments	--target /usr/sbin/psnet
aid	4294967295
binary	/bin/copyfail
cwd	/
docker	ae3126f3d30e4054eab60cea7161be3
exec_id	YWtZLWFhZW50G9vbc0zMjQzMzAzOS12bXNzMDAwMD8nOjE4NTMzOTg4NzUwNTY1NTg=
flags	execve rootcwd clone
in_init_tree	false
parent_exec_id	YWtZLWFhZW50G9vbc0zMjQzMzAzOS12bXNzMDAwMD8nOjE4NTMzOTg4NzUwNTY1NTg=
pid	56558
pod	({"container":{"id":"containerd://ae3126f3d30e4054eab60cea7161be39398a54a996df44d6b815ab0f53ca5b","image":{"id":"docker.io/copy-fail@sha256:48e61d8fa1448e9813a1b8a22fce03e"},"name":"copy-fail-poc","namespace":"tenant-blue"},"pod_labels":{"app.kubernetes.io/name":"copy-fail-poc","db6eb1d8-fa14-48e9-813a-1b8a22fce03e"},"workload":"copy-fail-poc"},"refcnt":1,"start_time":2026-05-14T15:47:18.1862700Z,"tid":56560,"uid":0,"return_action":"KPROBE_ACTION_POST"}
refcnt	1
start_time	2026-05-14T15:47:18.1862700Z
tid	56560
uid	0
return_action	KPROBE_ACTION_POST

Figure 9: Log relative to the AF_ALG socket.

5 Forensic considerations

Finally, we can draw some forensic considerations on the work done.

Kernel-Level Security Tetragon leverages eBPF to intercept and capture security events directly inside the Linux Kernel Space at the exact moment of system call invocation or through LSM hooks. By operating at this privileged layer, Tetragon inherently mitigates Time-of-Check to Time-of-Use (TOC-TOU) vulnerabilities [22], as parameters are inspected after being copied into kernel memory, preventing asynchronous userspace manipulation. Furthermore, because the tracking hooks reside entirely within the kernel domain, attackers operating from compromised or unprivileged containers lack the structural capabilities to tamper with, detach, or modify the instrumentation hooks or the generated logs, ensuring absolute visibility even during a root-level container breakout.

Enriched Forensic Context Rather than producing raw, isolated system call traces, Tetragon correlates low-level kernel events with rich, multi-dimensional metadata to provide deep forensic value. It automatically maps kernel-level operations to Kubernetes-aware identifiers, including container IDs, pod names, namespaces, and labels, allowing security teams to trace threats instantly back to specific workloads. Simultaneously, it maintains an internal process tree, capturing the accurate Process ID (PID), Thread ID (TID), parent process execution path (PPID), and the precise user/group

credentials (UID/GID) under which the binary was executed. This comprehensive context eliminates the need for manual log correlation during incident response investigations.

Real-Time Streaming & Non-Repudiation To minimize the critical window of exposure during an active breach, Vector acts as an almost real-time data pipeline that instantly streams Tetragon’s eBPF event logs directly into Log Analytics. Furthermore, the log ingestion pipeline enforces data non-repudiation and immutability: the target Azure Log Analytics workspace can be configured under an Append-Only architecture governed by Azure Resource Manager (ARM) locks and Immutable Storage policies, ensuring that once a log is transmitted by Vector, it can neither be modified, falsified, nor prematurely deleted by any entity within the cluster.

Pipeline Integrity & Trust The architectural pipeline guarantees end-to-end cryptographic integrity and strict access control from the cluster origin to the cloud SIEM. Vector utilizes robust TLS 1.2/1.3 to encrypt all payloads in transit, effectively neutralizing Man-in-the-Middle eavesdropping, tampering, or packet injection attempts. To authorize data ingestion, the pipeline leverages Microsoft Entra ID Workload Identities, authenticating the specific Vector agent service account. This zero-trust authentication mechanism ensures that only trusted cluster agents can authenticate against the Log Analytics API, completely preventing unauthorized external actors or rogue workloads from poisoning the workspace with spoofed logs.

6 Conclusions

In this work, we have understood how it can be difficult to detect the exploit of the Copy Fail vulnerability as it does not leave any traces on disk. Default monitoring tools given by the cloud providers are useful to understand the status or the resource usage of our cluster, but fail when it is necessary to identify the actual operations executed by someone in a container. Hence, we have analyzed how eBPF-based tools like Tetragon can help us in collecting richer logs that can be used in an investigation with a level of confidence similar to the one of default cloud provider logs.

References

- [1] Microsoft Azure Developers. Azure documentation. <https://learn.microsoft.com/en-us/azure/?product=popular>. [Accessed 19-05-2026].
- [2] Juno Im. Copy Fail: 732 Bytes to Root on Every Major Linux Distribution. <https://xint.io/blog/copy-fail-linux-distributions>. [Accessed 19-05-2026].
- [3] Percival and Cursor Agent. Percivalll/Copy-Fail-CVE-2026-31431-Kubernetes-PoC. <https://github.com/Percivalll/Copy-Fail-CVE-2026-31431-Kubernetes-PoC>. [Accessed 19-05-2026].
- [4] Juno Im. Copy Fail: From Pod to Host. <https://xint.io/blog/copy-fail-pod-to-host#scenario-1-crosscontainer-poisoning-1>. [Accessed 19-05-2026].
- [5] Petr Zuzanov. CVE-2026-31431: how Copy Fail behaves in Kubernetes. <https://www.stream.security/post/cve-2026-31431-how-copy-fail-behaves-in-kubernetes#>. [Accessed 19-05-2026].
- [6] Angela Wilson. The Copy Fail: Linux Kernel Local Privilege Escalation. <https://www.extrahop.com/blog/linux-kernel-local-privilege-escalation>. [Accessed 19-05-2026].
- [7] Microsoft Azure Developers. Azure Monitor overview. <https://learn.microsoft.com/en-us/azure/azure-monitor/fundamentals/overview>. [Accessed 19-05-2026].
- [8] Grafana Developers. Grafana | Query, visualize, alerting observability platform. <https://grafana.com/grafana/>. [Accessed 19-05-2026].
- [9] eBPF Developers. eBPF - Introduction, Tutorials & Community Resources. <https://ebpf.io>. [Accessed 19-05-2026].
- [10] Jed Salazar and Natalia Reka Ivanko. *Security Observability with eBPF*. O'Reilly Media, Inc., 2022.
- [11] Tetragon Developers. Tetragon - eBPF-based Security Observability and Runtime Enforcement. <https://tetragon.io>. [Accessed 19-05-2026].
- [12] Tetragon Developers. Welcome to Tetragon documentation. <https://tetragon.io/docs/>. [Accessed 19-05-2026].

- [13] Dean Lewis. Isovalent Enterprise for Tetragon: Deeper Host Network Observability with eBPF. <https://isovalent.com/blog/post/tetragon-network-observability-dashboards/>. [Accessed 19-05-2026].
- [14] Tetragon Developers. Tetragon - eBPF-based Security Observability & Runtime Enforcement. <https://isovalent.com/blog/post/2022-05-16-tetragon/#h-what-is-tetragon>. [Accessed 19-05-2026].
- [15] Thomas Graf. Tetragon 1.0: Kubernetes Security Observability & Runtime Enforcement. <https://isovalent.com/blog/post/tetragon-release-10/>. [Accessed 19-05-2026].
- [16] Jeremy Colvin. eBPF Security Observability: Top Tetragon Use Cases (Part 1). <https://isovalent.com/blog/post/top-tetragon-use-cases/#h-generate-audit-trails-for-mission-critical-applications>. [Accessed 19-05-2026].
- [17] Jeremy Colvin. eBPF Runtime Security at Scale: Top Tetragon Use Cases (Part 2). <https://isovalent.com/blog/post/top-tetragon-use-cases-part-2/>. [Accessed 19-05-2026].
- [18] Tetragon Developers. Tracing Policy. <https://tetragon.io/docs/concepts/tracing-policy/>. [Accessed 19-05-2026].
- [19] Vector Developers. Vector. <https://vector.dev>. [Accessed 19-05-2026].
- [20] Microsoft Azure Developers. Tutorial: Send data to Azure Monitor Logs with Logs ingestion API (Resource Manager templates). <https://learn.microsoft.com/en-us/azure/azure-monitor/logs/tutorial-logs-ingestion-api?tabs=dcr#create-azure-ad-application>. [Accessed 19-05-2026].
- [21] Jacqueline Tong. "Copy Fail" Linux kernel LPE and container escape - Information Security at University of Toronto. <https://security.utoronto.ca/advisories/copy-fail-linux-kernel-lpe-and-container-escape/>. [Accessed 19-05-2026].
- [22] Common Weakness Enumeration Community. CWE - CWE-367: Time-of-check Time-of-use (TOCTOU) Race Condition (4.17). <https://cwe.mitre.org/data/definitions/367.html>. [Accessed 19-05-2026].